n = 19 tau6 native per-shift solve, protocol v2 (run 39376405326: invalid)

2026-10-20

n = 19 tau6 native per-shift solve, protocol v2 (run 39376405326: invalid)

Status (2026-10-20): run, invalid. The Tier 1 run 39376405326 passed S0 and the field check but all 64 timed IC rows crashed, so there is no ratio. See Result. This protocol replaces v1, whose Tier 1 run 39307409409 was invalid because its S0 self-test failed. v1's and v2's protocols and archives are unchanged. The workflow `n19-native-tau6-v2.yml` has `workflow_dispatch` as its only trigger on `main`, so committing never launches a run. The owner decides when to dispatch it.

Why v1 failed, and what v2 changes

The full diagnosis is in `diagnosis/README.md`.

- **Cause.** v1's `arm.sage` binding read `run.sage`'s `MODULUS`, whose coefficients run lowest degree first, as highest degree first. It passed the reciprocal polynomial (829707) instead of Sage's modulus (854355). The native solve and its brute force share the context, so they agreed with each other on 300 of 300 systems and with Sage on 293.
- **Not affected.** `tau.sage`, the baseline row and `lib.rs`'s solve are correct.

	v1	v2
binding	inside `arm.sage`, which only Sage can load; modulus reversed	`native_binding.py`, plain Python, also run by the local checks; modulus lowest degree first
field check	none	in every native process, after its intervals: `tau6_modulus` = Sage's `F.modulus()`, and 256 `tau6_mul` products = Sage's
S0	300 systems; Sage vs native, native vs its own brute force; counts only	the 7 v1 mismatch systems + 300 new ones (Sage, native, brute force and direct S3 in Sage's field) + all 2,375 multi-solution systems (Sage order = native order); every system recorded
variety order	ascending (u, w), matches Sage on 176 of 2,375 multi-solution systems	ascending (w, u), Sage's order (2,375 of 2,375); ordered decompositions gated B = N = Naa = NX
`cargo test`	3 tests	7: adds the 7 systems, the v1 failure reproduced over the reciprocal, 15 recorded Sage orders, and the product through the C ABI
panel	`n19-native-heldout-v1`	`n19-native-heldout-v2`: new seed; also excludes the v1 panel
arm order, bootstrap	`Random(2026101919 + i)`, `default_rng(20261019)`	`Random('...-v2-20261019:order:%d')`, `default_rng(20261020)`
trigger	push (path filter) and dispatch	dispatch only

Everything else is v1's design, unchanged. This covers the question, the arms B, N, Naa, NX and rho, the intervals, the primary R = mean cold CPU(B) / mean cold CPU(N) with its paired bootstrap, the decision rule, the A/A window [0.95, 1.05], the planted lower bound > 1, O1 and the per-point gates. See `protocol.json` and v1's README for the derivation.

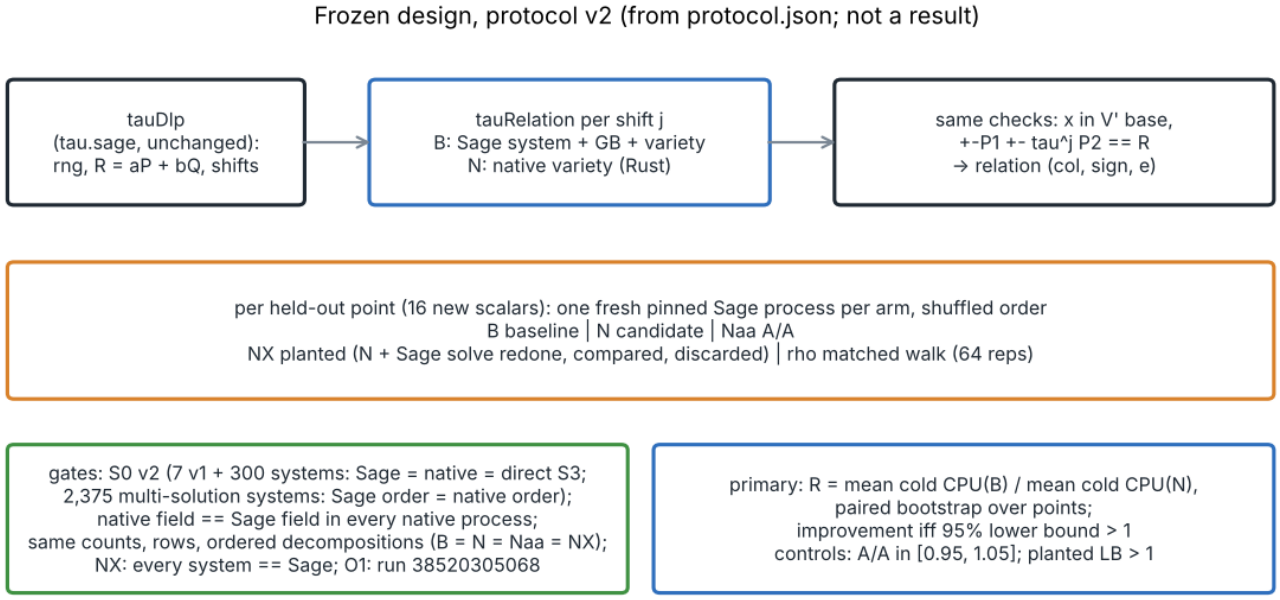


Figure 1: Frozen v2 design (not a result). Source: make_figures.py; vector: figures/design.svg.

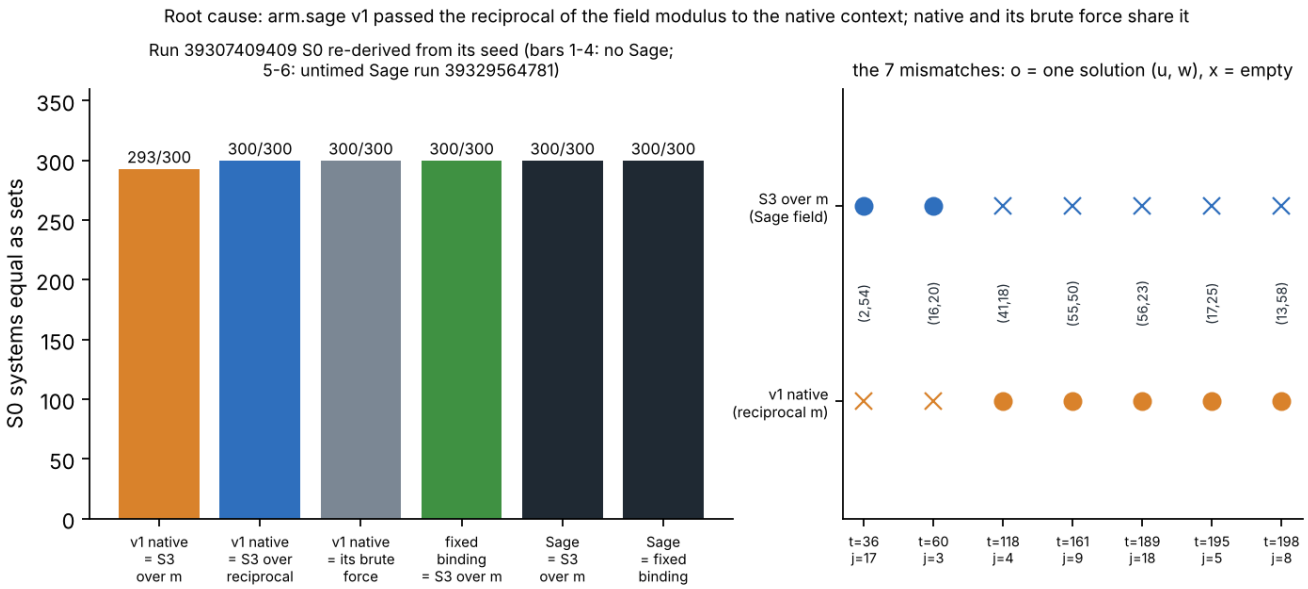


Figure 2: Run 39307409409's S0, re-derived without Sage. Vector: figures/s0_diagnosis.svg.

Order of a multi-solution variety (new in v2)

Sage's `variety()` returns a multi-solution variety ascending in (w, u). The untimed Sage diagnostic run 39329564781 recorded this on all 2,375 such systems of the field. v1 returned ascending (u, w), which matches Sage on only 176 of them.

- **Why it now matters.** v1's panel met only shift-0 swaps, where the order is harmless. The v2 panel meets three non-swap systems, and Sage's order differs from (u, w) on two of them. B and N would have returned different relations there.
- **The change.** v2's native solve writes the variety in Sage's order. The enumeration is unchanged.
- **The new gates.** The ordered decompositions must be identical across B, N, Naa and NX. S0 re-checks the order on all 2,375 systems on the runner. NX must agree in order on every multi-solution call.

See `diagnosis/README.md`.

Local checks before freezing (cloud-agent VM, no Sage; not timing evidence)

- `cargo test --release --locked`. 7 of 7 pass, locally and on the diagnostic runner.
- **Sage, untimed** (run 39329564781, `diagnosis/sage_run_39329564781/`). On the 307 v2 S0 systems, Sage = native = direct S3 on 307 of 307 (4 non-empty). `diagnosis/order_check.json` checks the order: the v2 library's output equals Sage's recorded order on 2,375 of 2,375 multi-solution systems and on all 607 S0 systems.
- **S0 without Sage.** `diagnosis/s0_v2_local.py` runs the v2 binding on the 307 v2 S0 systems. Native, its brute force and the pure-Python direct S3 agree on 307 of 307 (4 non-empty, 0 multi). The field check passes, and with the v1 modulus it fails on 256 of 256 products (`diagnosis/s0_v2_local.json`).
- **O1 and the panel without Sage.** `smoke/local_v2.py` runs `tau.sage`'s control flow in pure Python with the v2 binding. On the ten archived scalars it reproduces run 38520305068's tau6 targets, `gb_calls`, `spurious`, relations and logs on 10 of 10. On the 16 v2 points all logs verify, with a mean of 5,653 solve calls. Across both parts the candidate meets 36 multi-solution calls. 4 of them, on points p09, p10 and p11, hit 3 distinct non-swap systems (`smoke/local_v2.json`).
- **Power** (`analysis/power.json`, same model as v1, seed 20261020). With 16 points, power is 1.00 at a true ratio of 1.20, and the rejection rate with no effect is 0.035.

Result (run 39376405326)

Verdict: invalid. No primary statistic exists, and the n = 19 baseline row is unchanged.

- **Run.** 39376405326, push event on `claude/n19-native-solve` at `c432d184c`, GitHub ubuntu-24.04 (AMD EPYC 7763, image 20261012.334.1), one runner. The schedule step ran 43 min (05:45 to 06:28 UTC). The job conclusion is `success`; that only means the audit step ran. The launcher exited 0, and the audit reports `invalid`.
- **Archive.** `run_github_39376405326/` (artifact digest `sha256:b187660f...f5ec`). Re-running `verify_receipt.py` on a copy of the receipt (`runs.jsonl`, rows and logs) reproduces `panel/result.json` and `panel/result_table.md` byte for byte.
- **Why invalid.** All 82 scheduled rows ran; none was skipped by the time guard. S0 and O1 exited 0 and the 16 rho rows exited 0. **All 64 timed IC rows (16 points x B, N, Naa, NX) exited 1** with the same error, before writing a result: `TypeError: Object of type Integer is not JSON serializable`, raised in `arm.sage`'s `relation_digests` while hashing `rows` (`row.get(col, 0)` and the sum under Sage's preparser). The ordered and canonical hashes, built from `int(...)`, did not fail. This is the receipt's traceback; the preparser explanation is an inference from it, not tested here. v2's local pure-Python smoke test has no preparser, so it could not meet it.
- **Gates.** 37 of 118 pass.
  - **Pass.** S0: Sage = native = brute force, and Sage's order = native's, on all 2,682 systems (7 v1, 300 new, 2,375 multi-solution; direct S3 = native on 307 of 307; 4 evaluators agree). O1: counts and logs of run 38520305068 reproduced on all 10 scalars. Field check: native modulus and 256 products equal Sage's in every native process. One pinned CPU, one native digest (`d92534fa...87f2`), 64 rho reps verified on each of 16 points, ordered decompositions (vacuous: no rows), `cargo test` 7 of 7.
  - **Fail.** The scheduled-rows gate, and for each of the 16 points the counts and log, relation rows, NX-versus-Sage, generator and warm-batch gates (5 x 16 = 80), all because the rows are missing.
- **Controls.** A/A ratio N/Naa (window [0.95, 1.05]) and planted NX/N lower bound > 1 have no data.
- **Primary.** R = mean cold CPU(B) / mean cold CPU(N), and its 95% paired bootstrap interval: not computed. No improvement is claimed.
- **Secondaries.** None exist: online `S`, setup, the repeated-batch warm secondary, `S` per arm, the gap to the matched rho and the solve's share were not measured. Online `S` is separable by definition (above) and is still

unmeasured.

- **Tier 1 qualification.** Whatever the verdict, this protocol has one process per arm per point, not the repeated alternating pairs AGENTS.md's Tier 1 CPU isolation gate requires. A completed run would not have been a qualified Tier 1 speedup on that criterion.
- **What the run does show.** S0, with the order gate, passes on the full 2,682 systems, and the field check passes: the v1 binding fault and the order fault are fixed. The remaining fault is in `arm.sage`'s digest code.
- **Not claimed.** This run is not evidence that the native solve is faster, slower, or wrong in production.
- **Next.** Fixing `relation_digests` changes a frozen source (`arm.sage`), so it needs a new protocol (v3). This run is not re-run, and `protocol.json`, `schedule.json` and the sources are untouched.

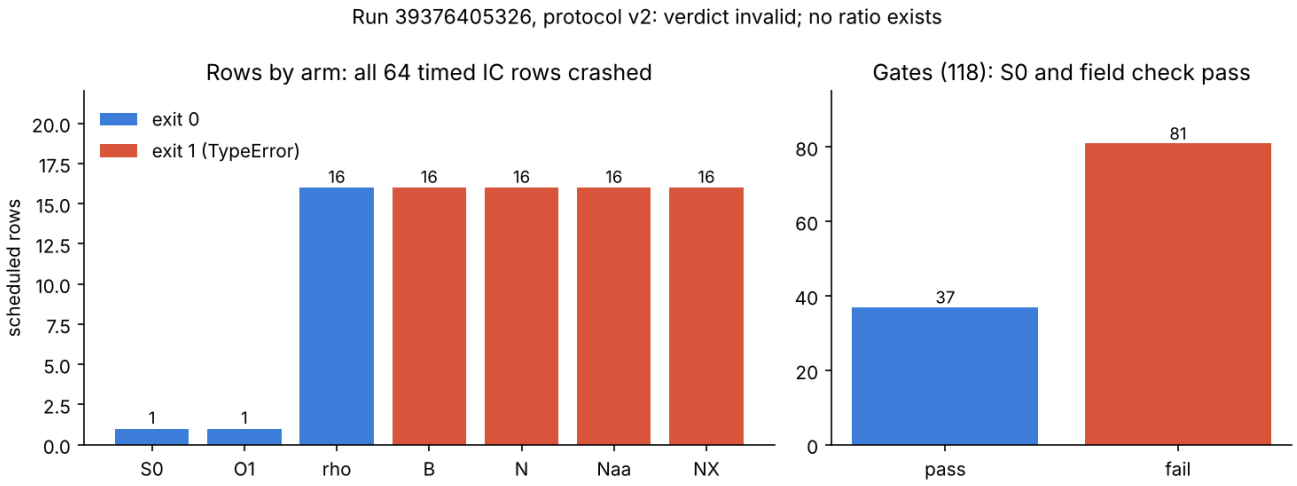


Figure 3: Rows and gates of run 39376405326. Source: make_result_figure.py; vector: figures/result.svg.

Files

- **Code.** `native/` (solve and tests), `native_binding.py`, `arm.sage`, `run_schedule.py`, `verify_receipt.py` and `stats.py`.
- **Frozen inputs.** `make_protocol.py` → `protocol.json`; `make_panel.py` → `inputs/panel.json`; `make_schedule.py` → `schedule.json`; `power_check.py` → `analysis/power.json`.
- **Diagnosis.** `diagnosis/`. The untimed Sage diagnostic is `diagnosis/sage_diag.sage`, run by `.github/workflows/n19-native-s0-diagnostic.yml`.
- **Timed workflow.** `.github/workflows/n19-native-tau6-v2.yml` (dispatch only).
- **Report.** `make_figures.py`, `make_result_figure.py`, `figures/`, `README.pdf` and `report_header.tex`.